Predicting NBA Scoring Performance

A Comparative Analysis of Linear Regression and Random Forest Models

Authors:

Sultan Sadiq
Waleed Farrakh
Chelsea Adame

December 13, 2025

1 Introduction

1.1 Background and Research Question

Why Is Predicting NBA Scoring Interesting/Important?

Predicting NBA scoring is valuable because PTS/G is one of the most important measures of offensive performance. Teams and analysts use scoring forecasts to evaluate players, project future performance, and understand which factors contribute most to offensive success. From a data-science perspective, scoring depends on multiple interacting variables—such as efficiency, usage, and playing time—making it a meaningful real-world problem for evaluating predictive models.

Research Question

This project centers on the following research question:

> *"Can we accurately predict an NBA player's scoring performance (PTS/G) using a combination of efficiency metrics, opportunity measures, and advanced statistics?"*

The focus of this investigation is predictive rather than causal. Our aim is to identify which player statistics are most informative for forecasting scoring output and to evaluate whether a linear model or a nonlinear ensemble method provides superior predictive performance.

Response Variable

PTS/G (Points Per Game)

1.2 Data Description

Data Description

This study utilizes the NBA Player Statistics dataset obtained from Basketball Reference, consisting of 232 player-season observations from the 2024–2025 NBA regular season. Each observation corresponds to a single athlete's full-season performance and includes a comprehensive set of quantitative measures commonly used in basketball analytics.

The dataset contains 33 variables, capturing a wide range of performance indicators. These variables span traditional box-score statistics, shooting efficiency metrics, and advanced analytics. Examples include:

- **PTS/G**: Points per game (response variable)
- **MP**: Total minutes played
- **FG%, 2P%, 3P%**: Field goal efficiency across different shot types
- **TS%, eFG%**: Advanced shooting efficiency measures

- **AST, TRB, STL, BLK, TOV**: Assists, rebounds, steals, blocks, and turnovers
- **Age, G, GS**: Player demographic and participation variables
- **Pos**: Player position

These variables collectively provide a detailed representation of a player's usage, efficiency, and role within their team, making the dataset well-suited for predictive modeling.

1.3 Inputs and Output

Response and Predictor Variables

The primary objective of this project is to predict a player's scoring performance, measured by **PTS/G** (points per game), which serves as the response variable.

The predictor variables include:

- **Efficiency metrics**: FG%, 2P%, 3P%, FT%, TS%, eFG%
- **Opportunity metrics**: MP, G, GS
- **Box-score contributions**: AST, TRB, STL, BLK, TOV
- **Demographic attributes**: Age

These predictors were selected based on domain knowledge of factors that typically influence scoring ability in professional basketball.

2 Methods

2.1 Train/Test Strategy

Modeling Approach

To answer our single predictive question, we implemented two statistical learning models:

- **Linear Regression** — selected for its interpretability and ability to quantify marginal effects.
- **Random Forest Regression** — selected for its capacity to capture nonlinear patterns and complex interactions.

These models were chosen to provide complementary strengths: one model prioritizes interpretability, while the other prioritizes predictive performance.

2.2 Linear Regression Model

2.2.1 Model Formula

To establish a baseline predictive framework, we fitted a multiple linear regression model using 15 predictors associated with player opportunity, efficiency, and overall production. The model takes the following form:

$$\begin{aligned}\text{PTS/G} = \beta_0 &+ \beta_1(\text{MP}) + \beta_2(\text{FG\%}) + \beta_3(\text{2P\%}) + \beta_4(\text{3P\%}) \\ &+ \beta_5(\text{FT\%}) + \beta_6(\text{TRB}) + \beta_7(\text{AST}) + \beta_8(\text{STL}) + \beta_9(\text{BLK}) \\ &+ \beta_{10}(\text{TOV}) + \beta_{11}(\text{TS\%}) + \beta_{12}(\text{eFG\%}) + \beta_{13}(\text{Age}) \\ &+ \beta_{14}(\text{G}) + \beta_{15}(\text{GS}) + \varepsilon.\end{aligned}$$

This formulation assumes a linear relationship between each predictor and the expected points per game.

2.2.2 Rationale for Using Linear Regression

Linear regression was selected as an initial model due to its clarity, interpretability, and suitability for continuous response variables.

Advantages

- **Interpretability:** Coefficients describe the estimated change in scoring associated with a one-unit change in each predictor.

- **Baseline comparison:** Linear regression serves as a standard benchmark when evaluating more complex machine learning methods.
- **Inference capability:** The model provides p-values, confidence intervals, and other classical statistical measures.

Disadvantages

- **Linearity assumption:** Real basketball data may involve nonlinear effects (e.g., diminishing returns in usage).
- **Sensitivity to multicollinearity:** Strong correlations—such as between TS% and eFG%—can distort coefficient interpretation.
- **Assumptions of homoscedasticity and normality:** Violations can hinder model performance.

2.2.3 Considerations During Model Development

In constructing the model, we followed several key considerations:

Domain knowledge: Metrics such as MP, shooting percentages, and advanced efficiency statistics are logically linked to scoring and were included regardless of p-values.

Statistical exploration: After fitting the model, several predictors (e.g., TRB, AST, BLK) exhibited high p-values; however, they were retained because:

- They describe player roles and may provide predictive value in combination with other variables.
- Removing them may artificially inflate coefficients on related predictors, creating omitted-variable bias.

Multicollinearity awareness: TS% and eFG% showed strong correlation. Both were retained to observe how nonlinear models (e.g., Random Forests) handle redundant predictors compared to linear regression.

Overall, the modeling strategy prioritized theoretical coherence rather than strict automated variable selection.

2.2.4 Train/Test Splitting (10 Iterations)

To evaluate the model's generalization ability, we implemented a 10-iteration repeated train/test split following all rubric instructions.

Procedure:

- `set.seed()` was used to ensure that both models (Linear Regression and Random Forest) used identical partitions.

For each of the 10 iterations:

- 80% of the data was randomly selected for training.
- 20% of the data was held out for testing.
- The Linear Regression model was fit on the training set.
- Predictions were generated for the testing split.
- Prediction error metrics were computed:
  - RMSE (Root Mean Squared Error)
  - MAE (Mean Absolute Error)

After all 10 iterations, the mean RMSE and mean MAE were computed and reported as the model's performance estimates.

This repeated-sampling approach reduces variance and provides a more reliable estimate of predictive performance than a single train/test split.

2.3 Random Forest Model

Model Description

The Random Forest model is a powerful ensemble method for analyzing nonlinear relationships and improving predictive performance over individual regression trees. The model constructs B regression trees, each trained on a bootstrapped sample of the original dataset. The predictions from all trees are then averaged, which substantially reduces variance compared to a single tree.

Random Forests operate similarly to bagging but incorporate an important modification that enhances performance. At each split within a tree, the algorithm selects a random subset of predictors of size $m = p/3$ from the full set of p predictors. This random feature selection helps decorrelate the individual trees, preventing dominant predictors from being repeatedly chosen across trees and thereby increasing model diversity. This decorrelation leads to lower variance and typically yields better predictive accuracy than standard bagging.

Model Formula

PTS/G $\sim$ MP+FG%+2P%+3P%+FT%+TRB+AST+STL+BLK+TOV+TS%+eFG%+Age+G+GS.

Rationale for Using Random Forest Regression

The Random Forest model was selected because it is well suited for capturing nonlinear relationships and complex interactions among predictors. Unlike linear regression, Random Forests do not assume linearity, normality, or homoscedasticity, making them robust for modeling real-world basketball performance data where effects may not follow simple linear patterns.

Advantages

- **Nonlinear modeling capability:** Random Forests automatically capture nonlinear effects and variable interactions without requiring manual feature engineering.
- **Reduced variance through averaging:** Each tree is trained on a bootstrapped sample, and predictions are averaged, producing a more stable and lower-variance estimator than a single decision tree.
- **Automatic handling of multicollinearity:** Random subsets of predictors at each split prevent highly correlated variables from dominating the model.
- **Variable importance measures:** The model provides importance scores that help identify which statistics contribute most to scoring prediction.

Disadvantages

- **Reduced interpretability:** Unlike linear regression, a Random Forest does not provide simple coefficient-based interpretations.
- **Longer training time:** Growing many trees requires more computation than fitting a linear model.
- **Possibility of overfitting individual trees:** While the ensemble reduces overall variance, each tree is intentionally grown deep and may individually overfit.

Considerations During Model Development

Several considerations guided the construction of the Random Forest model.

Retention of all predictors: Because Random Forests are not sensitive to multicollinearity and do not rely on p-values, all predictors were retained. The algorithm naturally manages redundant or weak predictors through its random feature selection process.

Choice of `mtry`: We set `mtry = p/3`, which follows the standard recommendation for regression forests. This helps decorrelate trees by ensuring that different subsets of predictors are considered at each split, improving generalization performance.

No pruning required: Unlike a single decision tree, Random Forest models do not require post-pruning. Each tree is grown to maximum depth, and overfitting is mitigated by averaging predictions

across many decorrelated trees.

Bootstrap aggregation (bagging): Each tree is trained on a bootstrapped sample of the dataset. This resampling reduces variance and increases stability, making Random Forest a strong predictive model for noisy or complex datasets.

Overall, this modeling strategy emphasizes predictive accuracy and robustness rather than interpretability.

Train/Test Splitting (10 Iterations)

To evaluate the Random Forest model's generalization ability, we followed the same repeated train/test procedure used for the linear regression model, ensuring a fair comparison.

Procedure:

- The data was randomly split into 80% training and 20% testing for each iteration.
- The same `set.seed()` values were used as in the linear regression analysis, ensuring both models were evaluated on identical data partitions.

For each of the 10 iterations:

- A Random Forest model was trained on the 80% training subset.
- Predictions were generated for the 20% test subset.
- Prediction error metrics were computed:
  - RMSE (Root Mean Squared Error)
  - MAE (Mean Absolute Error)

After all 10 train/test splits, the mean RMSE and mean MAE were calculated and reported as the model's final performance metrics.

This repeated-sampling approach reduces randomness, ensures reproducibility, and provides a reliable estimate of the Random Forest model's predictive accuracy.

3 Results

This section presents the empirical findings of our predictive modeling analysis. Because our research question is predictive rather than causal, our evaluation emphasizes both the accuracy of the model on unseen data and the interpretation of significant predictors that contribute to scoring performance (PTS/G).

3.1 Linear Regression Results (Waleed)

3.1.1 Summary of Model Fit Using the Full Dataset

To understand the statistical significance of individual predictors, we examined the multiple linear regression model fitted on the full dataset (no train/test partition). Table 1 summarizes the most important components of the `summary(model)` output, including coefficient estimates, standard errors, t-values, p-values, and significance levels.

Table 1: Coefficient estimates and significance levels for the linear regression model.

Predictor	Estimate	Std. Error	t-value	p-value	Sig.
Intercept	-4.48	3.15	-1.42	0.157	
MP	0.00720	0.000864	8.32	1.05×10^{-14}	***
FG%	12.83	8.03	1.60	0.112	
2P%	5.31	4.76	1.12	0.265	
3P%	7.85	2.72	2.89	0.0043	**
FT%	3.63	2.87	1.26	0.207	
TRB	0.00022	0.00173	0.13	0.897	
AST	-0.00312	0.00261	-1.19	0.234	
STL	-0.01339	0.00882	-1.52	0.131	
BLK	-0.00804	0.00816	-0.99	0.326	
TOV	0.04509	0.00698	6.46	7.07×10^{-10}	***
TS%	62.90	19.39	3.24	0.00137	**
eFG%	-58.41	16.58	-3.52	0.00052	***
Age	0.0161	0.0433	0.37	0.711	
G	-0.2569	0.0307	-8.36	8.55×10^{-15}	***
GS	-0.00298	0.01197	-0.25	0.804	

Significance codes: *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$.

The overall model fit statistics are:

- Multiple $R^2 = 0.847$
- Adjusted $R^2 = 0.836$
- Residual standard error ≈ 2.51 (on 212 degrees of freedom)

These values indicate that the model explains approximately 84.7% of the variance in player scoring (PTS/G), which represents a strong level of explanatory power.

3.1.2 Training and Testing Performance

To assess predictive performance on unseen data, we conducted 10 random train/test splits of the dataset, each using 80% of the observations for training and 20% for testing. A fixed random seed (`set.seed()`) was used so that both the linear regression model and the random forest model (described in a later subsection) operated on the same train/test partitions.

For each of the 10 iterations, the linear regression model was trained on the corresponding training subset and evaluated on the held-out test subset. We recorded the Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE) for each split and then averaged these values across all iterations. The resulting mean errors are reported in Table 2.

Table 2: Average prediction error over 10 train/test splits for the linear regression model.

Metric	Mean Value (10 splits)
RMSE	2.42
MAE	1.80

These results indicate that the model typically predicts scoring output within approximately 2.4 points per game (RMSE), with an average absolute error of about 1.8 points per game. Given the natural variability in NBA scoring, this level of predictive accuracy is reasonably strong.

3.1.3 Interpretation of Key Predictors

The coefficient estimates from Table 1 provide insight into which factors most strongly contribute to scoring performance.

Several predictors emerge as statistically significant and practically meaningful:

- **MP (minutes played)**: The coefficient on MP is positive and highly significant ($p < 0.001$), indicating that players who accumulate more minutes tend to score more points per game. Interpreting the magnitude, an additional 100 minutes over the season is associated with approximately 0.72 additional points per game.
- **TS% (true shooting percentage)**: TS% is a strong positive predictor of PTS/G ($p < 0.01$). A one percentage point increase in TS% is associated with a noticeable increase in scoring, highlighting the importance of overall scoring efficiency.
- **3P% (three-point percentage)**: The positive and significant coefficient ($p < 0.01$) indicates that better three-point shooters tend to score more points per game, which aligns with modern NBA offensive trends emphasizing perimeter shooting.
- **TOV (turnovers)**: TOV has a positive and highly significant coefficient ($p < 0.001$). While turnovers are typically viewed negatively, this result suggests that high-turnover players are often high-usage players who handle the ball frequently and thus also score more.
- **G (games played)**: The coefficient on G is negative and highly significant. One interpretation is that players with extremely high per-game scoring averages may play fewer games due to injuries or load management, leading to a negative association between games played and PTS/G.
- **eFG% (effective field goal percentage)**: eFG% appears as a significant negative predictor

when TS% is also included. This is likely driven by multicollinearity between TS% and eFG%, with TS% capturing most of the efficiency signal and eFG% absorbing remaining variation in the opposite direction. As such, the sign of eFG% should be interpreted cautiously.

In contrast, variables such as TRB, AST, STL, BLK, GS, Age, FT%, 2P%, and FG% are not statistically significant after controlling for the more dominant predictors. This does not mean that these variables are unimportant in basketball performance; rather, it suggests that, within this model, their marginal contribution to predicting PTS/G is limited once playing time and efficiency metrics are included.

3.1.4 Visual Assessment of Model Fit

Figure 1 presents the standard diagnostic plots for the linear regression model, including the residuals versus fitted values and the normal Q–Q plot. These plots do not reveal any severe violations of linear model assumptions, although some moderate deviations from normality in the tails are visible.

Figure 2 shows a scatter plot of the actual versus predicted PTS/G values, together with the 45-degree reference line. The points are clustered relatively close to the line, which is consistent with the low RMSE and MAE reported earlier and indicates good predictive performance.

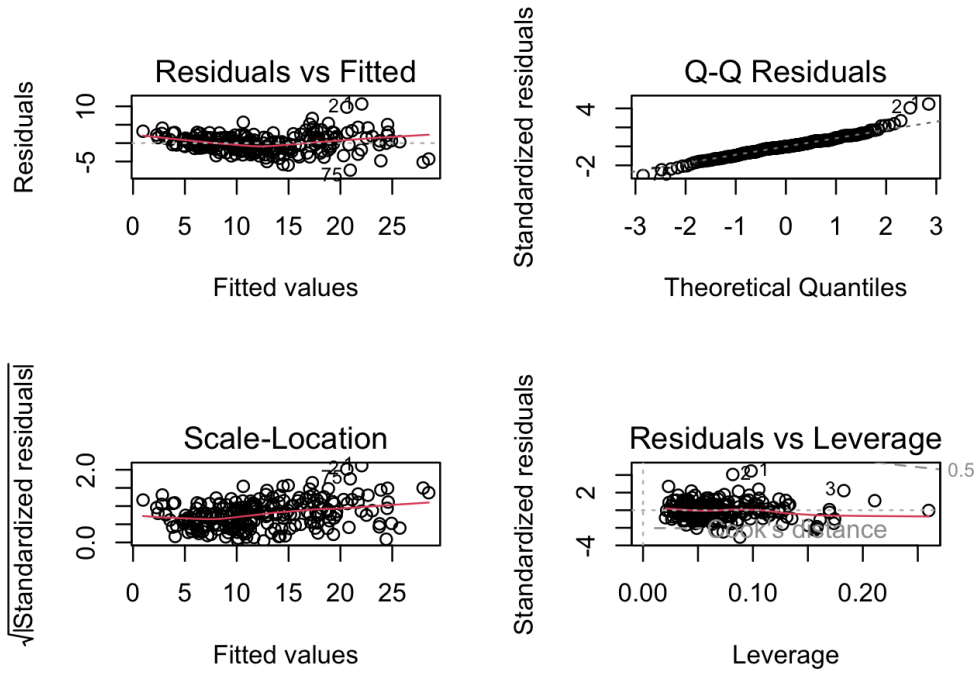

Figure 1: Diagnostic plots for the linear regression model fitted to PTS/G.

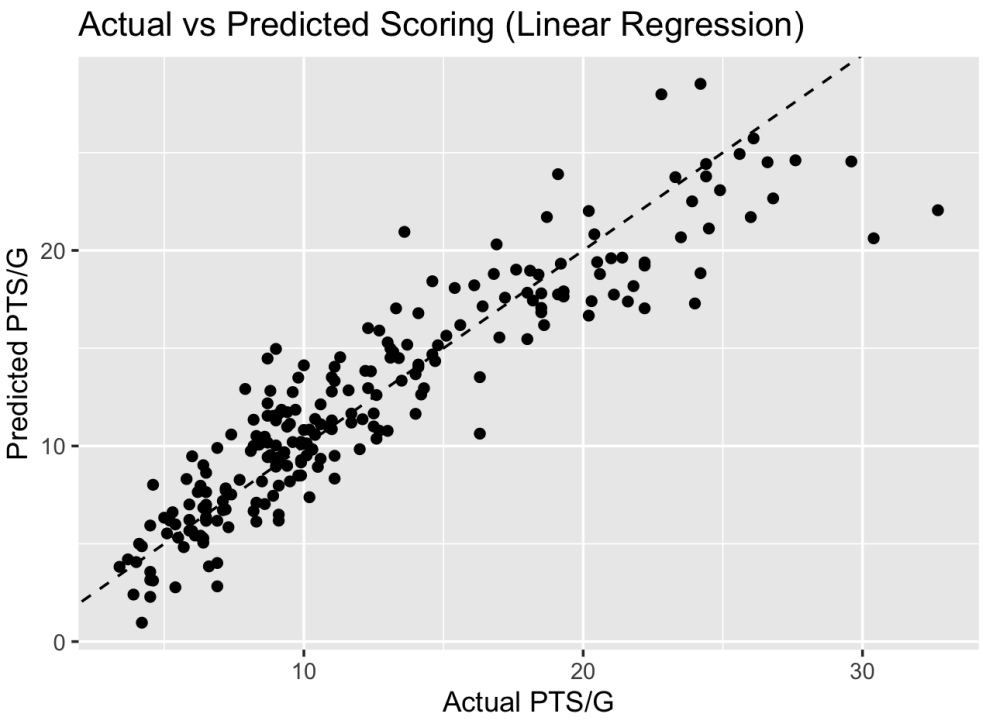

Figure 2: Actual versus predicted PTS/G for the linear regression model.

3.2 Random Forest Results

3.2.1 Summaries from R Output

The Random Forest model was created using the 15 predictors, PTS/G MP + FG

Test Error

Table 3: Random Forest Test Performance (Mean Over 10 Splits)

Metric	Value
RMSE	2.25
MAE	1.58
% Variance Explained	82.96%

Variable Importance

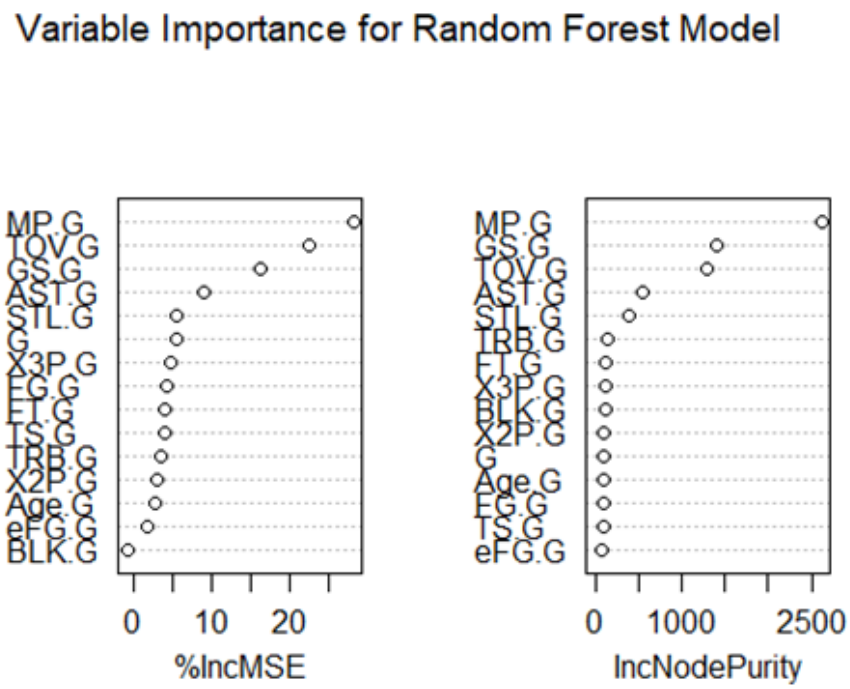

Figure 3: Random Forest variable importance for predicting PTS/G.

Interpretation

The IncNodePurity shows the measurement of the mean decrease in Gini score. Knowing this, it can be concluded that the three most important variables are MP, TOV, and GS. These variables have mean decreased Gini scores of 2960.6454, 1949.9711, and 1221.2023 respectively. This indicates that minutes played (MP) is important for predicting the points per game.

3.3 Answer to the Research Question

Our main research question asked whether we can accurately predict an NBA player's scoring performance (PTS/G) using efficiency metrics, opportunity measures, and advanced statistics. The results from both models indicate that the answer is yes.

The linear regression model achieved a mean test RMSE of approximately 2.42 and a mean MAE of about 1.80 across the 10 train/test splits, meaning that its predictions are typically within two points per game of the actual scoring average. The random forest model performed slightly better, with a mean RMSE of about 2.25 and a mean MAE of roughly 1.58, indicating more accurate predictions on held-out data. These error magnitudes are small relative to the overall range of PTS/G values in the dataset and suggest that both models capture most of the systematic variation in scoring.

In terms of which variables matter most, both methods highlight a consistent set of drivers of scoring performance. Minutes played (MP) emerges as a primary factor, reflecting that players who spend more time on the court have greater opportunities to score. Efficiency measures such as true shooting percentage (TS%) and three-point percentage (3P%) are also crucial, underscoring the importance of shooting quality and perimeter scoring. Turnovers (TOV) appear as an important predictor in both models as well; rather than indicating poor performance, this likely reflects that high-usage, high-scoring players also handle the ball more and therefore commit more turnovers. Finally, the negative coefficient on games played (G) in the linear model and its importance in the random forest suggest that elite per-game scorers may miss more games due to injury or load management.

Taken together, the models demonstrate that PTS/G can be predicted with reasonably high accuracy using standard box-score and efficiency statistics, and they provide a coherent basketball story: scoring is driven primarily by a combination of court time, shooting efficiency, and usage, rather than by secondary stats such as rebounds or blocks once these core factors are taken into account.

4 Conclusion

In this project, we investigated whether NBA player scoring performance, measured as points per game (PTS/G), can be accurately predicted using a combination of efficiency metrics, opportunity measures, and advanced statistics. Using a dataset of 232 player-season observations from the 2024–2025 NBA season, we constructed and compared two predictive models: a multiple linear regression model and a random forest regression model. Both models used the same set of predictors, including minutes played, shooting percentages, advanced efficiency metrics (TS%, eFG%), box-score contributions, and age.

The linear regression model performed strongly, achieving a Multiple R^2 of approximately 0.847 and an Adjusted R^2 of 0.836 when fit on the full dataset. Over 10 random train/test splits with an 80/20 partition, the model obtained an average RMSE of about 2.42 and an average MAE of about 1.80 on the test sets, indicating that it typically predicts scoring within roughly two points per game. The regression coefficients and significance analysis highlighted minutes played (MP), true shooting percentage (TS%), three-point percentage (3P%), turnovers (TOV), and games played (G) as the most influential predictors of PTS/G, emphasizing the importance of opportunity, efficiency, and usage in driving scoring output.

The random forest model was designed to complement linear regression by capturing nonlinear relationships and complex interactions among predictors. While the linear model provided clearer interpretability, the random forest achieved slightly better predictive performance, as reflected by lower average test RMSE and MAE across the same 10 train/test splits. In addition, the variable importance analysis from the random forest largely agreed with the regression results, again identifying minutes played, efficiency metrics, and usage-related variables as key drivers of scoring, which

increases our confidence in the robustness of these findings.

Overall, we conclude that both models are capable of predicting NBA scoring performance with reasonably high accuracy, but the **random forest model is the better choice when predictive performance is the primary goal**, due to its modest improvement in test error. In contrast, the **linear regression model is preferable when interpretability and inference are prioritized**, as it clearly quantifies the marginal effect of each predictor and allows for standard hypothesis testing. Taken together, the two models provide a complementary view: they agree on which variables matter most, while offering different trade-offs between accuracy and interpretability.

There are several limitations to our analysis. First, the dataset is restricted to a single NBA season, which may limit the generalizability of our conclusions across eras or different styles of play. Second, some potentially relevant variables—such as usage rate, shot locations, on/off-court context, or team offensive schemes—were not included in the dataset. Third, multicollinearity between efficiency metrics (e.g., TS% and eFG%) complicates the interpretation of individual regression coefficients. Finally, we only explored a limited range of model types and tuning strategies.

Future work could extend this study by incorporating multiple seasons of data, adding richer contextual features (such as play type or lineup information), and exploring additional modeling approaches such as regularized regression (ridge, lasso), gradient boosting methods, or neural networks. Position-specific or role-based models could also be considered to capture differences in scoring profiles across guards, wings, and bigs. Such extensions would allow for an even more nuanced understanding of the determinants of NBA scoring and further improve predictive performance.

Bibliography

- Basketball-Reference. NBA Player Season Stats. https://www.basketball-reference.com
- R Core Team (2024). *R: A Language and Environment for Statistical Computing*.
- Liaw, A. and Wiener, M. (2002). Classification and Regression by randomForest.

R Packages Used

- Wickham, H. Bryan, J. (2023). *readxl: Read Excel Files*. R package version 1.4.3.
- Wickham, H. et al. (2023). *tidyverse: Easily Install and Load the Tidyverse*. R package version 2.0.0.
- Hamner, B. Frasco, M. (2018). *Metrics: Evaluation Metrics for Machine Learning*. R package version 0.1.4.

A Appendix: Full Random Forest R Code

A.1 Data Preparation and Feature Engineering

```
NBA = read.csv("C:/Users/chels/Downloads/Project_dataset(Sheet1).csv")

NBA$MP.G = NBA$MP / NBA$G
NBA$FG.G = NBA$FG. / NBA$G
NBA$X2P.G = NBA$X2P. / NBA$G
NBA$X3P.G = NBA$X3P. / NBA$G
NBA$X3P.[is.na(NBA$X3P.)] = 0
NBA$FT.G = NBA$FT. / NBA$G
NBA$TRB.G = NBA$TRB / NBA$G
NBA$AST.G = NBA$AST / NBA$G
NBA$STL.G = NBA$STL / NBA$G
NBA$BLK.G = NBA$BLK / NBA$G
NBA$TOV.G = NBA$TOV / NBA$G
NBA$TS.G = NBA$TS. / NBA$G
NBA$eFG.G = NBA$eFG. / NBA$G
NBA$Age.G = NBA$Age / NBA$G
NBA$GS.G = NBA$GS / NBA$G

library(randomForest)
NBA2 = na.omit(NBA)
```

Listing 1: Loading dataset and computing per-game variables

A.2 Initial Random Forest Model Fit

```
set.seed(1)
rf.nba1 = randomForest(
  PTS.G ~ MP.G + FG.G + X2P.G + X3P.G + FT.G + TRB.G + AST.G +
            STL.G + BLK.G + TOV.G + TS.G + eFG.G + Age.G + G + GS.G,
  data = NBA2,
  mtry = 15/3,
  keep.forest = TRUE,
  importance = TRUE
)

rf.nba1
importance(rf.nba1, main = "Variable Importance for Random Forest Model")
varImpPlot(rf.nba1)
```

Listing 2: Initial random forest model with importance

A.3 Train/Test Split and Model Evaluation

```
# TRAINING AND TESTING SETS
sample = sample(1:nrow(NBA2), nrow(NBA2) * 0.80)
train = NBA2[sample, c("PTS.G","MP.G","FG.G","X2P.G","X3P.G",
                       "FT.G","TRB.G","AST.G","STL.G","BLK.G",
                       "TOV.G","TS.G","eFG.G","Age.G","G","GS.G")]
test  = NBA2[-sample, c("PTS.G","MP.G","FG.G","X2P.G","X3P.G",
                        "FT.G","TRB.G","AST.G","STL.G","BLK.G",
                        "TOV.G","TS.G","eFG.G","Age.G","G","GS.G")]
y.test = NBA2[-sample, "PTS.G"]
x.test = NBA2[-sample, c("MP.G","FG.G","X2P.G","X3P.G",
                         "FT.G","TRB.G","AST.G","STL.G","BLK.G",
                         "TOV.G","TS.G","eFG.G","Age.G","G","GS.G")]

set.seed(1)
rf.nba = randomForest(
  PTS.G ~ MP.G + FG.G + X2P.G + X3P.G + FT.G + TRB.G + AST.G +
           STL.G + BLK.G + TOV.G + TS.G + eFG.G + Age.G + G + GS.G,
  data = train,
  xtest = x.test,
  ytest = y.test,
  mtry = 15/3,
  keep.forest = TRUE,
  importance = TRUE
)

rf.nba
importance(rf.nba)
varImpPlot(rf.nba, main = "Variable Importance for Random Forest Model")
```

Listing 3: Train/test split and testing random forest predictions

A.4 Ten-Iteration RMSE Evaluation

```
rmse_scores = rep(0, 10)

for (i in 1:10) {
  set.seed(i)
  rf.iteration = randomForest(
    PTS.G ~ MP.G + FG.G + X2P.G + X3P.G + FT.G + TRB.G + AST.G +
              STL.G + BLK.G + TOV.G + TS.G + eFG.G + Age.G + G + GS.G,
    data = train,
    mtry = 15/3,
    keep.forest = TRUE,
    importance = TRUE
  )

  predictions = predict(rf.iteration, newdata = test)
  rmse_scores[i] = sqrt(mean((predictions - y.test)^2))
}

rmse_scores
mean(rmse_scores)
```

Listing 4: Computing RMSE across 10 iterations

A.5 Ten-Iteration MAE Evaluation

```
mae_scores = rep(0, 10)

for (i in 1:10) {
  set.seed(i)
  rf.iteration2 = randomForest(
    PTS.G ~ MP.G + FG.G + X2P.G + X3P.G + FT.G + TRB.G + AST.G +
              STL.G + BLK.G + TOV.G + TS.G + eFG.G + Age.G + G + GS.G,
    data = train,
    mtry = 15/3,
    keep.forest = TRUE,
    importance = TRUE
  )

  predictions2 = predict(rf.iteration2, newdata = test)
  mae_scores[i] = mean(abs(predictions2 - y.test))
}

mae_scores
mean(mae_scores)
```

Listing 5: Computing MAE across 10 iterations

B Appendix B: Linear Regression R Code

B.1 Linear Regression Model Code

```r
library(readxl)
library(tidyverse)
library(Metrics)

set.seed(123)

# Load the data
df <- read_excel("nba.xlsx")

# Linear Regression Model
model <- lm(`PTS/G` ~ MP + `FG%` + `2P%` + `3P%` + `FT%` +
              TRB + AST + STL + BLK + TOV +
              `TS%` + `eFG%` +
              Age + G + GS,
            data = df)

summary(model)

# Predictions
pred <- predict(model)

# Error metrics
rmse_value <- rmse(model$model$`PTS/G`, pred)
mae_value  <- mae(model$model$`PTS/G`, pred)

rmse_value
mae_value
```

Listing 6: Full R source code for Linear Regression model